

Chapter 29: Shor's Algorithm

Kamonluk Suksen Ph.D.

Background - Encryption

- Nowadays, data encryption is an indispensable part of our daily life.
- In the encryption process, we usually generate a very big integer number, N , using two large prime numbers, P and Q , such that $N = P \times Q$.
- N is shared with the public, and only the persons who have a key can find P and Q quickly.
- Otherwise, to break the encryption (e.g. in RSA), one needs to be able to perform **prime integer factorization** of N , i.e. to find the values of P and Q .
- There is no known method to do it quickly. Internet security relies almost entirely on the fact/expectation that no one can perform the prime integer factorization efficiently.

Background – Period Finding

- The prime integer factorization of N is related to the period finding of a certain function.
- More precisely, if we want to factorize the number N , mathematically, it is equivalent to finding the period, r , of the following function (we will not prove this):

$$f_{a,N}(x) \equiv a^x \bmod N$$

where a is a parameter and x is an independent variable. Therefore, $f_{a,N}(x)$ is a function depending on the parameters, a and N , and the variable x . The variable x is also an integer.

- This is a function using the **modulo operator**, $\bmod$, which returns the remainder of a division. For example, $7 \bmod 2 = 1$ because $7 = 3 \times 2 + 1$. When 7 is divided by 2, the remainder is 1.
- Therefore, the function returns the remainder of a^x divided by N .

Background – Period Finding

- For example, assume we want to factorize $N = 21$. Let us pick $a = 11$ (we will discuss how to pick a later.) Find the period, r , of the function $f_{a,N}(x) = f_{11,21}(x)$. We will find it by substituting x beginning from 0.

$$\begin{aligned}
 f_{11,21}(0) &\equiv 11^0 \bmod 21 = 1 & \text{as } 11^0 &= 0 \times 21 + 1 \\
 f_{11,21}(1) &\equiv 11^1 \bmod 21 = 11 & \text{as } 11^1 &= 0 \times 21 + 11 \\
 f_{11,21}(2) &\equiv 11^2 \bmod 21 = 16 & \text{as } 11^2 &= 5 \times 21 + 16 \\
 f_{11,21}(3) &\equiv 11^3 \bmod 21 = 8 & \text{as } 11^3 &= 63 \times 21 + 8 \\
 f_{11,21}(4) &\equiv 11^4 \bmod 21 = 4 & \text{as } 11^4 &= 697 \times 21 + 4 \\
 f_{11,21}(5) &\equiv 11^5 \bmod 21 = 2 & \text{as } 11^5 &= 7669 \times 21 + 2 \\
 f_{11,21}(6) &\equiv 11^6 \bmod 21 = 1 & \text{as } 11^6 &= 84360 \times 21 + 1
 \end{aligned}$$

- We see that the answer repeats with a period of 6 ($f_{11,21}(0) = f_{11,21}(0 + 6)$). Therefore, $r = 6$.
- Note that finding the period is computationally intensive.

Background – Prime Integer Factorization

- Once we find the period r of $f_{a,N}(x)$, we can perform factorization in a quick way.
- Let us consider the following derivation.

Due to the periodicity, r ,

$$\begin{aligned} f_{a,N}(0) &= a^0 \bmod N = 1 \bmod N = 1 \\ f_{a,N}(r) &= f_{a,N}(r+0) = a^r \bmod N = f_{a,N}(0) = 1 \end{aligned}$$

where we used the fact that $1 \bmod N = 1$ (i.e. the remainder of 1 divided by N is 1 as long as $N > 1$). We can use the rules in algebra and rearrange the second equation, and we have

$$\begin{aligned} a^r - 1 \bmod N &= 0 \\ (a^{\frac{1}{2}r} - 1)(a^{\frac{1}{2}r} + 1) \bmod N &= 0 \end{aligned}$$

where we use the rule like in the algebra that $(a^{\frac{r}{2}} - 1)(a^{\frac{r}{2}} + 1) = a^r - 1$.

- Then we will use a theorem that *if r is even and $a^{\frac{r}{2}} + 1 \bmod N \neq 0$, then the prime factors are given by*

$$\begin{aligned} P &= \gcd(a^{\frac{r}{2}} + 1, N) \\ Q &= \gcd(a^{\frac{r}{2}} - 1, N) \end{aligned}$$

where $\gcd(x, y)$ refers to the greatest common divisor of x and y . For example, $\gcd(10, 20) = 10$.

Background – Prime Integer Factorization

- Another example is $\gcd(21, 14) = 7$ because 7 is the largest number and can divide (without remainder) 21 and 14.
- *If r is not even or $a^{\frac{r}{2}} + 1 \bmod N = 0$, we need to try another a .*
- There is a theory that says that 50% of the a will satisfy this requirement. Therefore, we have a high chance to be able to use the following equation to do factorization:

$$P = \gcd(a^{\frac{r}{2}} + 1, N)$$

$$Q = \gcd(a^{\frac{r}{2}} - 1, N)$$

- Therefore, once the period is found, the problem is converted into a problem of finding the $\gcd$ of the numbers that can be done effectively e.g. using the **Euclidean algorithm**.
- Let us continue on the previous example. We know that $r = 6$ now. Find P and Q .

$r = 6$ is even. We still need to check if $a^{\frac{r}{2}} + 1 \bmod N \neq 0$.

$$a^{\frac{r}{2}} + 1 \bmod N = 11^{\frac{6}{2}} + 1 \bmod 21 = 1332 \bmod 21 = 9 \neq 0$$

Therefore, we can go ahead to find the $\gcd$. We have

$$\gcd(a^{\frac{r}{2}} + 1, N) = \gcd(11^{\frac{6}{2}} + 1, 21) = \gcd(1332, 21) = 3$$

$$\gcd(a^{\frac{r}{2}} - 1, N) = \gcd(11^{\frac{6}{2}} - 1, 21) = \gcd(1330, 21) = 7$$

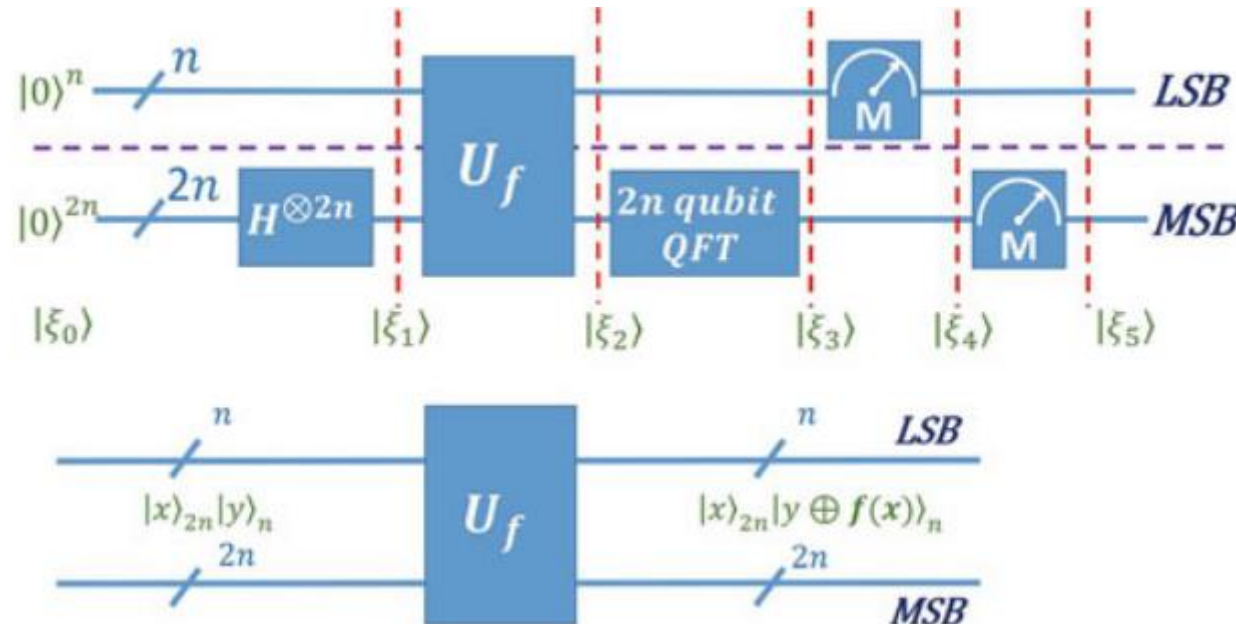
Therefore, $21 = 3 \times 7$.

Shor's Algorithm

- To break the classical encryptions, we need to perform prime integer factorization of a big number ($N = P \times Q$).
- We can do this by first finding the period, r , of the function $f_{a,N}(x) \equiv a^x \bmod N$.
- Once the period is found to be an even number and $a^{\frac{r}{2}} + 1 \bmod N \neq 0$, we can find P and Q easily by finding the *gcd* of some numbers.
- We need to guess the parameter a so we can use the theory, but the chance of picking the right a is pretty high (50%).
- The most time-consuming part of this procedure is in the **period finding** process. The quantum computing part of the **Shor's algorithm** is used to speed up this process.
- Classically, the period finding process complexity is $O(r)$ (refer to computational complexity in Chapter Grover's Algorithm). With Shor's algorithm, it becomes $O(\log(r))$ and thus it is an exponential speedup.

Shor's Algorithm

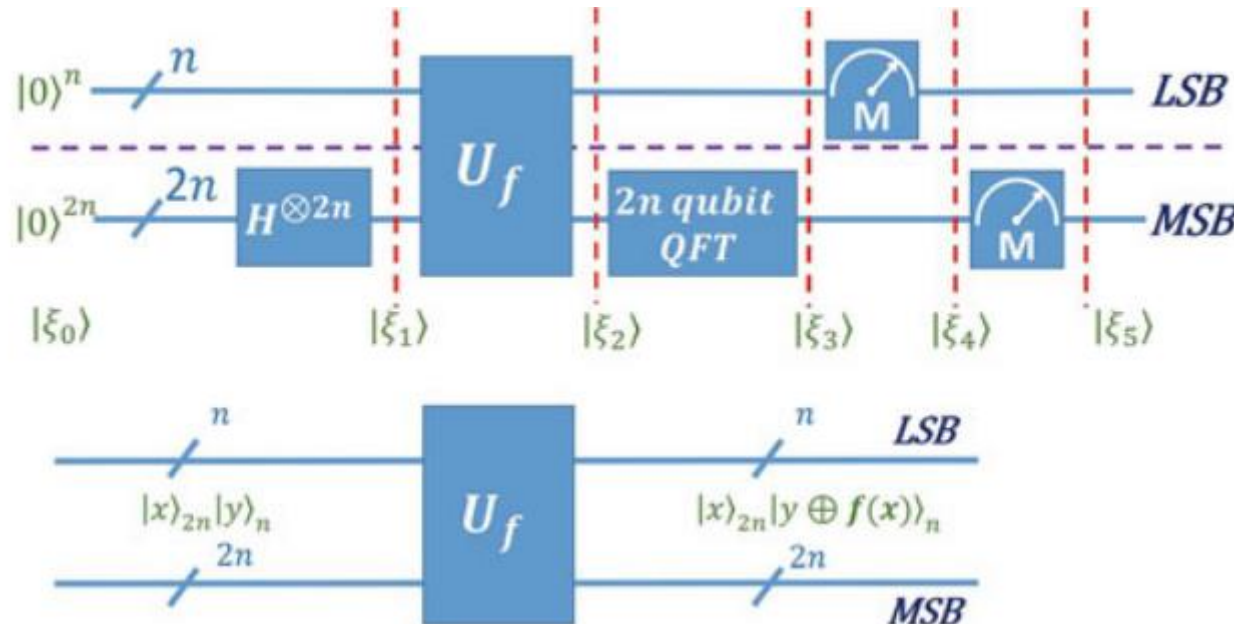
- Pic. 52 shows the quantum circuit of Shor's algorithm. There are 4 parts in Shor's algorithm.
For a function $f(x) = f(x + r)$:
 1. Create a superposition of exponentially many arguments.
 2. Evaluate the function on a superposition of exponentially many arguments.
 3. Compute a parallel Fourier transform on the superposition using QFT.
 4. Sample the Fourier power spectrum to obtain the function's period.



Pic. 52 Top: quantum circuit for Shor's algorithm. Bottom: XOR quantum oracle circuit displayed for convenience.

Shor's Algorithm

- For any N , we can always find n such that $2^n \geq N$. For example, if $N = 21$, we need $n = 5$ so that $2^n = 32 \geq 21$.
- In Shor's algorithm, we need $3n$ qubits. They are grouped into two registers. The register with the more significant qubits has $2n$ qubits, and the other one has n qubits as shown in Pic. 49.
- Therefore, the quantum state starts with $|\xi_0\rangle = |0\rangle^{\otimes 2n} |0\rangle^{\otimes n}$.



Pic. 52 Top: quantum circuit for Shor's algorithm. Bottom: XOR quantum oracle circuit displayed for convenience.

Shor's Algorithm

- **Step 1:** After that, a $2n$ -qubit Hadamard gate, $H^{\otimes 2n}$, is applied to the first register, while the second register is kept intact.

$$\begin{aligned}
 |\xi_1\rangle &= (H^{\otimes 2n} I^{\otimes n})(|0\rangle^{\otimes 2n} |0\rangle^{\otimes n}) \\
 &= (H^{\otimes 2n} |0\rangle^{\otimes 2n})(I^{\otimes n} |0\rangle^{\otimes n}) \\
 &= \left(\frac{1}{2^{\frac{2n}{2}}} \sum_{x=0}^{2^{2n}-1} |x\rangle_{2n} \right) \otimes |0\rangle^{\otimes n} \\
 &= \frac{1}{2^n} \sum_{x=0}^{2^{2n}-1} |x\rangle_{2n} |0\rangle_n
 \end{aligned}$$

- A n -qubit identity gate, $I^{\otimes n}$, is also applied at the same time to the last n -qubit register. We added parentheses to separate the operators and states in the first line. Then parentheses are used to separate the two registers in the second line.
- Each 1-qubit Hadamard gate transforms $|0\rangle$ into $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, and the products of the n transformed qubits become the summation in the third line. Note that the x in each $|x\rangle$ is written in decimal form, and it has $2n$ qubits as indicated in the subscript. This step creates a superposition of exponentially many basis states (2^{2n}) to be evaluated in parallel in the next step.

Shor's Algorithm

- **Step 2:** Since the Shor's algorithm involves a function $f_{a,N}(x) \equiv a^x \bmod N$, we naturally expect we need a quantum oracle that has the property of $f_{a,N}(x)$.

- The quantum oracle has $3n$ qubits. As shown in Pic. 49, it transforms any basis vector of the $3n$ qubits by

$$U_f |x\rangle_{2n} |y\rangle_n = |x\rangle_{2n} |y \oplus f(x)\rangle_n$$

which is just the regular behavior of an XOR quantum oracle. The most significant $2n$ qubits are the query qubits, and the least significant n qubits are the auxiliary qubits. The $f(x)$ here is the $f_{a,N}(x)$ function of which the period is to be found.

- Let us apply the quantum oracle to $|\xi_1\rangle$. We will use the above equation with $y = 0$.

$$\begin{aligned} |\xi_2\rangle &= U_f |\xi_1\rangle \\ &= U_f \left(\frac{1}{2^{2n}} \sum_{x=0}^{2^{2n}-1} |x\rangle_{2n} \right) |0\rangle_n \\ &= \frac{1}{2^n} \sum_{x=0}^{2^{2n}-1} |x\rangle_{2n} |0 \oplus f(x)\rangle_n \\ &= \frac{1}{2^n} \sum_{x=0}^{2^{2n}-1} |x\rangle_{2n} |f(x)\rangle_n \end{aligned}$$

- Note that the classical XOR operation of the label (i.e. $0 \oplus f(x)$) of the least significant n qubits is relatively straightforward because $y = 0$, and thus the least significant n qubits become just the corresponding values of $f(x)$. For example, if $n = 3$ and $f(x) = 5$, then $|0 \oplus f(x)\rangle_n = |5\rangle_3 = |101\rangle$.
- This step evaluates the exponentially many basis vectors with label x all at one time in parallel. That is, it evaluates $f(0)$ up to $f(2^{2n} - 1)$ in one execution. This is where the quantum computing provides the exponential speedup.

Shor's Algorithm

- **Step 3:** However, the evaluated results are stored as the basis vectors in a superposition wavefunction, and we cannot extract them directly. The next step is to perform QFT to extract the results we want. We apply a QFT operation defined in equation:

$$U_{QFT}|x\rangle = \frac{1}{\sqrt{2^{2n}}} \sum_{k=0}^{2^{2n}-1} \omega^{-kx} |k\rangle$$

- The QFT is only applied to the first register. We also need to apply an n -qubit identity gate to the second register. Therefore,

$$|\xi_3\rangle = (U_{QFT} \otimes I^{\otimes n}) |\xi_2\rangle$$

$$= (U_{QFT} \otimes I^{\otimes n}) \left(\frac{1}{2^n} \sum_{x=0}^{2^{2n}-1} |x\rangle_{2n} |f(x)\rangle_n \right)$$

$$= \frac{1}{2^n} \sum_{x=0}^{2^{2n}-1} (U_{QFT}|x\rangle_{2n}) (I^{\otimes n}|f(x)\rangle_n) = \frac{1}{2^n} \sum_{x=0}^{2^{2n}-1} (U_{QFT}|x\rangle_{2n}) (|f(x)\rangle_n)$$

$$= \frac{1}{2^n} \sum_{x=0}^{2^{2n}-1} \left(\frac{1}{\sqrt{2^{2n}}} \sum_{k=0}^{2^{2n}-1} (\omega^{-kx} |k\rangle_{2n}) (|f(x)\rangle_n) \right) = \frac{1}{2^{2n}} \sum_{x=0}^{2^{2n}-1} \sum_{k=0}^{2^{2n}-1} \omega^{-kx} |k\rangle_{2n} |f(x)\rangle_n$$

Shor's Algorithm

- **Step 4:** We will now perform a measurement on the least significant n -qubit (the second register). The wavefunction will collapse to one of the $|f(x)\rangle_n$. For example, if the function is $f_{11,21}(x)$, there are only 6 possible $|f(x)\rangle_n$, which are $|1\rangle$, $|11\rangle$, $|16\rangle$, $|8\rangle$, $|4\rangle$, and $|2\rangle$.
- However, there are many x that will give the same $f(x)$. For example, $f(x = 0) = f(x = 6) = f(x = 12) = \dots = 1$ and $f(x = 2) = f(x = 8) = f(x = 14) = \dots = 16$.
- And this is because of the periodicity of the function. Therefore, when the wavefunction collapses, for example becomes $|f(x_0)\rangle_n$ it actually collapses by keeping $|f(x_0)\rangle_n$, $|f(x_0 + r)\rangle_n$, $|f(x_0 + 2r)\rangle_n$, $|f(x_0 + 3r)\rangle_n$, etc., where r is the periodicity. That means when the wavefunction collapses to a certain state after measurement, it will keep all x separated by the period r . Therefore, if after the measurement, it collapses to $|f(x_0)\rangle_n$, the wavefunction becomes $|\xi_4\rangle$.

Shor's Algorithm

• Step 4:

$$\begin{aligned}
 |\xi_4\rangle &\approx \sum_{x=x_0+yr} \sum_{k=0}^{2^{2n}-1} \omega^{-kx} |k\rangle_{2n} |f(x)\rangle_n \\
 &= \sum_{x=x_0+yr} \sum_{k=0}^{2^{2n}-1} \omega^{-kx} |k\rangle_{2n} |f(x_0)\rangle_n \\
 &= \sum_{k=0}^{2^{2n}-1} \sum_{x=x_0+yr} \omega^{-kx} |k\rangle_{2n} |f(x_0)\rangle_n \\
 &= \sum_{k=0}^{2^{2n}-1} \sum_y \omega^{-k(x_0+yr)} |k\rangle_{2n} |f(x_0)\rangle_n \\
 &= \sum_{k=0}^{2^{2n}-1} \sum_y \omega^{-kx_0} \omega^{-kyr} |k\rangle_{2n} |f(x_0)\rangle_n
 \end{aligned}$$

In the first line, we removed all x not separated by multiples of r from x_0 . Therefore, the summation $\sum_{x=0}^{2^{2n}-1}$ becomes $\sum_{x=x_0+yr}$, which means we only have a periodic series of x that are $x_0, x_0 + r, x_0 + 2r$, etc. and y are integers. (If there were no periodicity, we even would not have this summation.) The maximum y should be such that are:

$x = x_0s + yr < 2^{2n} - 1$. *Note that we also skipped the constants in front of the summations.* This is because we need to perform normalization after the measurement and the equation is too messy to put.

In the second line, we just replace $|f(x)\rangle_n$ by $|f(x_0)\rangle_n$ because all x in $x = x_0 + yr$ give $|f(x_0)\rangle_n$.

In the fourth line, we then convert the summation of x into a summation of y and rewrite $x = x_0 + yr$ in the exponents.

Shor's Algorithm

- **Step 5:** We will now perform a measurement on the first register that has the most significant $2n$ qubits. Assume it will collapse to $|k_0\rangle$. Then

$$\begin{aligned} |\xi_5\rangle &\approx \sum_y \omega^{-k_0 x_0} \omega^{-k_0 y r} |k_0\rangle_{2n} |f(x_0)\rangle_n \\ &= \omega^{-k_0 x_0} \left(\sum_y \omega^{-k_0 y r} \right) |k_0\rangle_{2n} |f(x_0)\rangle_n \end{aligned}$$

where again we do not perform normalization and skip the constant coefficients.

- We see that the coefficient of the measured state is proportional to $\sum_y \omega^{-k_0 y r}$, and we put in a pair of parentheses to emphasize this. Let us substitute $\omega = e^{i2\pi/2^{2n}}$.

$$\sum_y \omega^{-k_0 y r} = \sum_y e^{-i2\pi k_0 y r / 2^{2n}}$$

- We will not evaluate the sum but will try to understand it graphically.

Shor's Algorithm

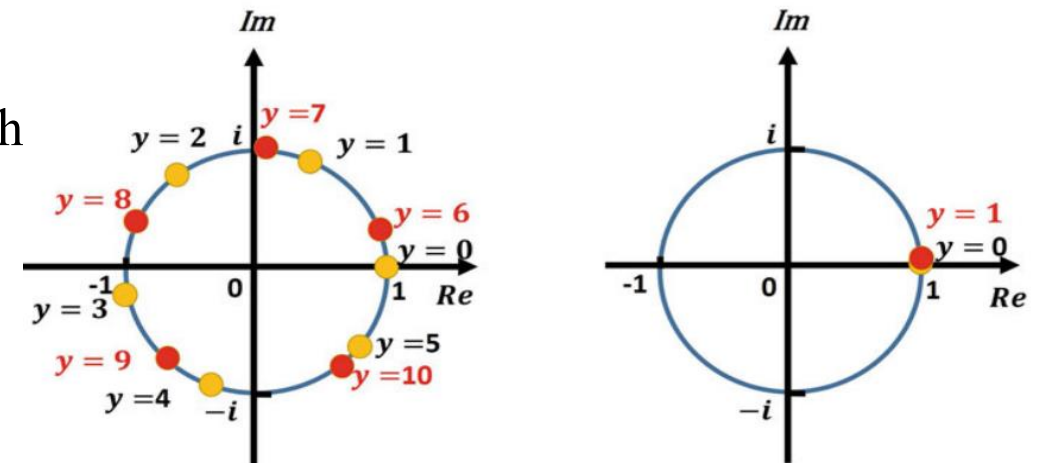
- **Step 5:** The constructive interference that is:

$$\frac{2\pi k_0 y r}{2^{2n}} = 2\pi d$$

$$r = 2^{2n} d / k_0$$

where d is an integer indicating how many circles it goes around when y increases by 1.

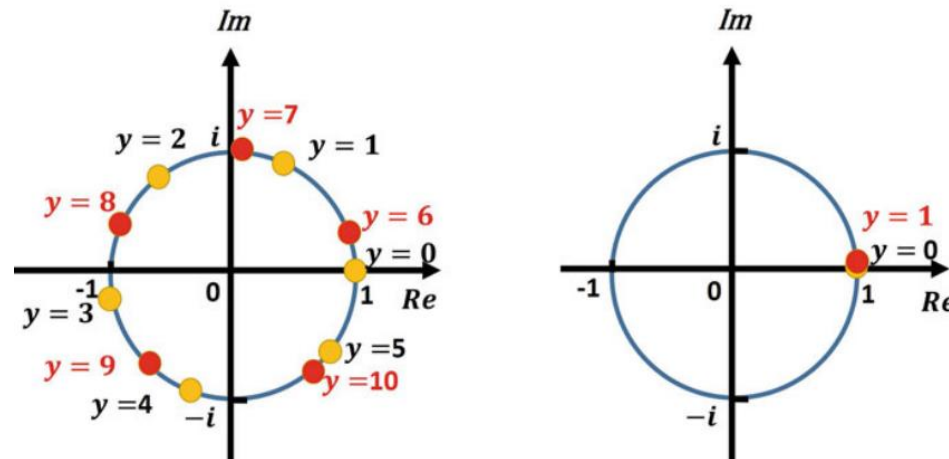
- Here, it means that if we measure $|k_0\rangle$, then the period r is $2^{2n} d / k_0$.
- Since d is an integer that we do not know, we still do not know the value of r .
- However, Shor's algorithm already helps to narrow down the possible values of r .
- After this, the classical **continued fraction expansion** will be used to further narrow down the possible value of r , which will not be discussed here. The result can be checked easily, and if it is wrong, we will redo Shor's algorithm until the correct answer is found.



Pic. 53 Plotting of $e^{-i2\pi k_0 y r / 2^{2n}}$ on the complex plane. Yellow dots refer to the first round, and red dots refer to the second round. Left: Distribution of the terms resulting in destructive interference. Right: Distribution of the terms resulting in constructive interference

Shor's Algorithm

- **Step 5:** Pic. 53 shows two types of possible distributions of $e^{-i2\pi k_0 y r / 2^{2n}}$.
- Starting with $y = 0$, the terms go around the unit circle in the complex plane anti-clockwise. After going through many rounds, the numbers distribute fairly evenly on the unit circuit. Their summation is thus very small as they cancel the others (**destructive interference**).
- This happens for some k , and thus the probability of measuring these $|k\rangle$ is very small and near zero. On the other hand, certain values of k will give a large enough and just enough step to go around a full circle when y is incremented by 1.
- In this case, all the terms will be at the same point or very close to the others. This means the phase of $e^{-i2\pi k_0 y r / 2^{2n}}$ needs to increase by 2π or multiples of 2π when y is increased by 1 so to result in a **constructive interference**.



Pic. 53 Plotting of $e^{-i2\pi k_0 y r / 2^{2n}}$ on the complex plane. Yellow dots refer to the first round, and red dots refer to the second round. Left: Distribution of the terms resulting in destructive interference. Right: Distribution of the terms resulting in constructive interference

Chapter 30: Harrow–Hassidim–Lloyd (HHL) Quantum Algorithm

Kamonluk Suksen Ph.D.

Introduction

- The HHL algorithm, proposed by Aram Harrow, Avinatan Hassidim, and Seth Lloyd in 2009, is used for solving linear systems of equations using the principles of quantum computing.
- To solve such a system, we cast our problem in the form $A|x\rangle = |b\rangle$, where $|x\rangle$ and $|b\rangle$ are normalized vectors and A is a hermitian matrix.
- The process involves finding the eigenvalues of the matrix by making use of the Quantum Phase Estimation (QPE) sub-routine. This in turn makes use of the inverse Quantum Fourier Transform (QFT).
- The determined eigenvalues are then used to implement a controlled rotation to effectively find the inverse of the matrix A . This allows us to calculate $|x\rangle = A^{-1}|b\rangle$. The final step is to uncompute the phase estimation.
- The Harrow-Hassidim-Lloyd (HHL) algorithm solves the Quantum Linear Systems Problem (QLSP) in $\mathcal{O}(\frac{K^2 S^2}{\epsilon})$ steps, where K is the condition number of the matrix, S is its sparsity (maximum non-zero entries per row), and ϵ is the desired solution accuracy. This is an exponential speed-up over the best classical methods, which scale as $\sim N^{2.81}$, provided that K and S remain small.
- This algorithm serves as a subroutine of more advanced algorithms such as those pertaining to Machine Learning and the modeling of quantum systems.

Mathematical Formulation

- Our problem is expressed as:

$$A|x\rangle = |b\rangle \quad (1)$$

Where A is an $N \times N$ Hermitian matrix, and $|x\rangle$ and $|b\rangle$ are normalized vectors. Using the spectral decomposition theorem, we can express A as:

$$A = \sum_{j=0}^{N-1} \lambda_j |u_j\rangle\langle u_j|, \quad (2)$$

- Since A is hermitian, its eigenstate is orthonormal. We can also calculate the inverse of A as:

$$A^{-1} = \sum_{j=0}^{N-1} \lambda_j^{-1} |u_j\rangle\langle u_j|. \quad (3)$$

- Further, we express $|b\rangle$ in the eigenstate of A as:

$$|b\rangle = \sum_{i=0}^{N-1} b_i |u_i\rangle. \quad (4)$$

- Now, the solution vector for Eq. (1) is given by: $|x\rangle = A^{-1}|b\rangle$.

Mathematical Formulation

- By substituting in equations (3) and (4) into the above, we get:

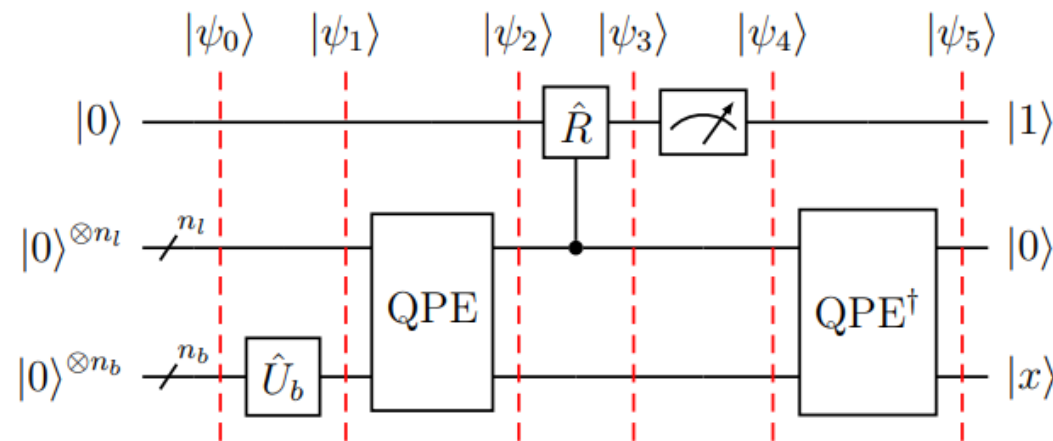
$$\begin{aligned}
 |x\rangle &= \left(\sum_{j=0}^{N-1} \lambda_j^{-1} |u_j\rangle\langle u_j| \right) \left(\sum_{i=0}^{N-1} b_i |u_i\rangle \right) \\
 &= \sum_{j=0}^{N-1} \sum_{i=0}^{N-1} \lambda_j^{-1} b_i |u_j\rangle\langle u_j|u_i\rangle \\
 |x\rangle &= \sum_{j=0}^{N-1} \lambda_j^{-1} b_j |u_j\rangle \quad (5)
 \end{aligned}$$

Note that $|x\rangle$ is normalized, hence:

$$\sum_{j=0}^{N-1} |\lambda_j^{-1} b_j|^2 = 1 \quad (6)$$

Tracing the Evolution of the State

- We can trace the evolution of the state in a step-wise fashion with reference to Pic. 54.
- The circuit contains three registers: the first register is based on n_b qubits, so that it can be encoded with the value of $|b\rangle$. The size of the $|b\rangle$ vector is thus $N \times 1$ where $N = 2^{n_b}$.
- The second register, also known as the clock register, is based on n_l qubits. This is meant to hold the eigenvalues of A .
- The final register is a single ancilla qubit. In the diagram, the convention followed is that the most significant bit is at the bottom and the least significant bit is at the top.



Pic. 54 Circuit diagram.

Tracing the Evolution of the State

- We initialize the state as:

$$|\psi_0\rangle = |0\rangle^{\otimes n_b} |0\rangle^{\otimes n_l} |0\rangle.$$

- We encode the information for $|b\rangle$ onto the n_b register using a unitary operation $\hat{U}_b$ such that $\hat{U}_b|0\rangle = |b\rangle$ to obtain the following:

$$|\psi_1\rangle = |b\rangle^{\otimes n_b} |0\rangle^{\otimes n_l} |0\rangle.$$

- Next, we apply the Quantum Phase Estimation subroutine to encode the eigenvalues on to the n_l register. This results on the following state:

$$|\psi_2\rangle = |b\rangle^{\otimes n_b} |\tilde{\lambda}_j\rangle^{\otimes n_l} |0\rangle.$$

- We now want to find the inverse of the eigenvalues. To this end, we apply a controlled rotation on the ancilla qubit in the top register, based on the value of the n_l register. Thus we obtain following state:

$$|\psi_3\rangle = \sum_{j=0}^{N-1} b_j |u_j\rangle |\tilde{\lambda}_j\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} |0\rangle + \frac{C}{\lambda_j} |1\rangle \right).$$

- In the above equation, C is a normalization constant for the third register. The next step is to perform a measurement on the third register. The desired outcome is $|1\rangle$, which will result in the following state:

Tracing the Evolution of the State

$$|\psi_4\rangle = \frac{1}{\sqrt{\sum_{j=0}^{N-1} \left| \frac{b_j C}{\lambda_j} \right|^2}} \sum_{j=0}^{N-1} b_j |u_j\rangle |\tilde{\lambda}_j\rangle \left(\frac{C}{\lambda_j} |1\rangle \right).$$

where the pre-factor is there to ensure normalization post measurement. If the measurement had resulted in the $|0\rangle$ state, we would have repeated the procedure until we obtained the desired state. With a little manipulation, the state $|\psi_4\rangle$ becomes:

$$|\psi_4\rangle = \frac{C}{\sqrt{\sum_{j=0}^{N-1} \left| \frac{b_j C}{\lambda_j} \right|^2}} \sum_{j=0}^{N-1} b_j \lambda_j^{-1} |u_j\rangle |\tilde{\lambda}_j\rangle |1\rangle.$$

- If C is real, we can cancel it out; secondly, we make use of the normalization condition from Eq.(6) to obtain the following simplified expression:

$$|\psi_4\rangle = \sum_{j=0}^{N-1} b_j \lambda_j^{-1} |u_j\rangle |\tilde{\lambda}_j\rangle |1\rangle.$$

Tracing the Evolution of the State

- We then enter the uncompute stage, where we apply the inverse phase estimation, hence returning the n_l register to the $|0\rangle$ state. The state now becomes:

$$|\psi_5\rangle = \sum_{j=0}^{N-1} b_j \lambda_j^{-1} |u_j\rangle |0\rangle^{\otimes n_l} |1\rangle.$$

- Now we know from Eq. (5) that $|x\rangle = \sum_{j=0}^{N-1} b_j \lambda_j^{-1} |u_j\rangle$, so:

$$|\psi_5\rangle = |x\rangle |0\rangle^{\otimes n_l} |1\rangle.$$

- We can see that the final state $|\psi_5\rangle$ is proportional to our solution vector.

Implementation Methodology

- In this section, we explain the gate level implementation of the HHL algorithm and present a worked example simulated on IBM qiskit.
- Here we have chosen a 2×2 hermitian matrix A such that:

$$A = \begin{pmatrix} \frac{3}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{3}{4} \end{pmatrix},$$

and the following $|b\rangle$ vector:

$$|b\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

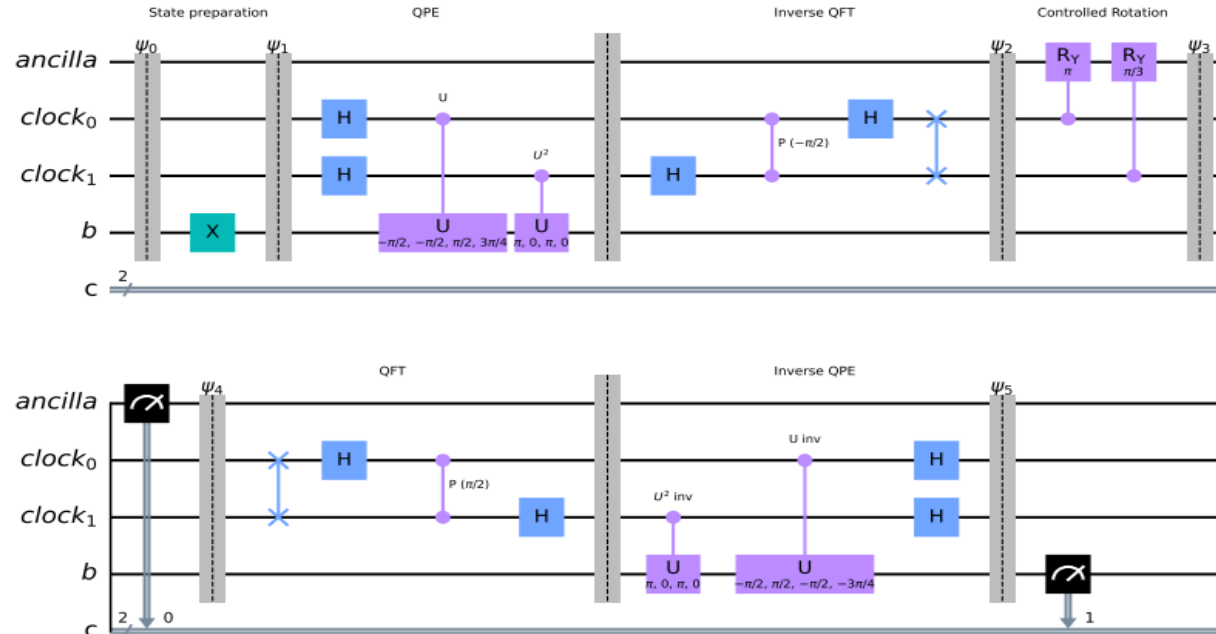
- The solution vector for this system is $x = \begin{pmatrix} -\frac{1}{2} \\ \frac{3}{2} \end{pmatrix}$, and the ratio of the squares of the magnitudes of its components is $1 : 9$ which we will later compare with the results of the simulation to verify its accuracy.

Implementation Methodology

- The eigenvectors for A are $|U_0\rangle = \begin{pmatrix} \frac{-1}{\sqrt{2}} \\ 1 \end{pmatrix}$ and $|U_1\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 1 \end{pmatrix}$, whereas its eigenvalues are $\lambda_0 = \frac{1}{2}$ and $\lambda_1 = 1$.

Using this information, we can diagonalize A as follows:

$$A = P\Lambda P^{-1} = \frac{1}{2} \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix}$$



Pic. 55 Simulated circuit on IBM Qiskit

Implementation Methodology

1. Encoding $|b\rangle$

- We assume that there exists an operator that can efficiently encode the vector $|b\rangle$ onto the first register, or that $|b\rangle$ is the output of a different algorithm which feeds into our circuit.
- In our example, where $b = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, the application of a simple NOT gate will suffice.

2. Hamiltonian Simulation

- In quantum mechanics, when given a Hamiltonian H , we can write the time evolution of a quantum state as $|\psi(t)\rangle = e^{-iHt/\hbar}|\psi(0)\rangle$, where $e^{-iHt/\hbar}$ is a unitary operator.
- We can treat matrix A which is one of the main inputs of the HHL algorithm as a Hamiltonian, the only precondition being that A is hermitian, as is the case for a true Hamiltonian. However, quantum gates can only implement unitary operations. Thus, we modify our hermitian matrix to form $U = e^{-iAt}$, which is unitary.
- There are multiple ways to implement this operator at a gate level, depending on the complexity of the Hamiltonian. For our numerical example involving a single qubit vector $|b\rangle$, U can be implemented on Qiskit using the following four-parameter gate, with the appropriate choice of input parameters:

$$U_{\theta\phi\gamma\lambda} = \begin{pmatrix} e^{i\gamma} \cos \frac{\theta}{2} & -e^{i(\gamma+\lambda)} \sin \frac{\theta}{2} \\ e^{i(\gamma+\phi)} \sin \frac{\theta}{2} & e^{i(\gamma+\lambda+\phi)} \cos \frac{\theta}{2} \end{pmatrix}$$

- Any single-qubit unitary operation can be carried out using the above operator, and it is trivial to modify it to form a control U gate for use in QPE:

$$U_c = U \otimes |1\rangle\langle 1| + I \otimes |0\rangle\langle 0|.$$

Implementation Methodology

3. Quantum Phase Estimation

- A unitary matrix acting on its eigenvector is represented as follows:

$$U|u\rangle = e^{i2\pi\phi}|u\rangle.$$

- If the unitary matrix is of the form $U = e^{iAt}$, with A being a Hermitian matrix bearing the eigenvalues λ_i , then the above equation can be written as:

$$e^{iAt}|u\rangle = e^{i\lambda_i t}|u\rangle.$$

- Comparing the two equations, we obtain:

$$\phi_i = \frac{\lambda_i t}{2\pi}.$$

- If the input vector is not an eigenvector of the unitary operator, it can be expressed as a linear combination of the eigenvectors, and due to the linearity of operators, the transformation acts on each of these eigenvectors.
- The Quantum Phase Estimation (QPE) process acts on the clock register as $|0\rangle^{\otimes n_l} \rightarrow |\phi_i 2^{n_l}\rangle = |\widetilde{\lambda}_i\rangle$. Using $\phi_i = \frac{\lambda_i t}{2\pi}$, we can write the output of the QPE as:

$$|\widetilde{\lambda}_i\rangle = \frac{\lambda_i t 2^{n_l}}{2\pi}.$$

- In our example, $n_l = 2$ and, in order to obtain integer values, t is chosen to be π and $\lambda_0 = \frac{1}{2}$ and $\lambda_1 = 1$. Hence we obtain:

$$\begin{aligned} |\widetilde{\lambda}_0\rangle &= |1_{10}\rangle = |01_2\rangle, \\ |\widetilde{\lambda}_1\rangle &= |2_{10}\rangle = |10_2\rangle, \end{aligned}$$

- This is an instance of Basis Encoding, where the eigenvalue is mapped onto the computational basis of the qubits, which involves the use of the binary representation of the eigenvalues.

Implementation Methodology

3. Quantum Phase Estimation

- Next, given that A is a hermitian matrix we can exponentiate it to create a unitary operator $U = e^{iAt}$. We can determine the exact form of U by comparing with $A = P\Lambda P^{-1} = \frac{1}{2} \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix}$:

$$U = \frac{1}{2} \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} i & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix}$$

where the diagonal elements of the second matrix are obtained by:

$$e^{i\lambda_0 t} = e^{i\frac{\pi}{2}} = i.$$

$$e^{i\lambda_1 t} = e^{i\pi} = -1.$$

- Hence:

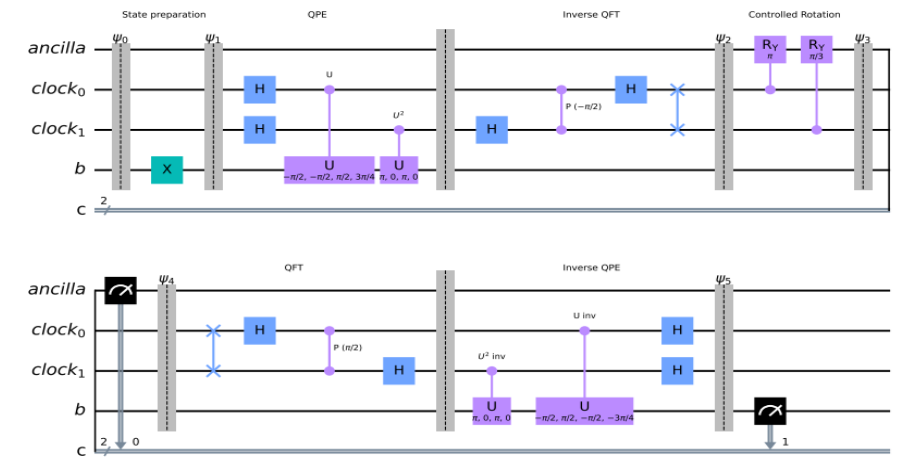
$$U = \frac{1}{2} \begin{pmatrix} -1 + i & -1 - i \\ -1 - i & -1 + i \end{pmatrix},$$

and

$$U^2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

- In order to implement U and U^2 in the simulated circuit in Pic. 55, we make use of the four parameter arbitrary unitary gate from:

$$U_{\theta\phi\gamma\lambda} = \begin{pmatrix} e^{i\gamma} \cos \frac{\theta}{2} & -e^{i(\gamma+\lambda)} \sin \frac{\theta}{2} \\ e^{i(\gamma+\phi)} \sin \frac{\theta}{2} & e^{i(\gamma+\lambda+\phi)} \cos \frac{\theta}{2} \end{pmatrix} \text{ by setting } \theta = -\frac{\pi}{2}, \phi = -\frac{\pi}{2}, \lambda = \frac{\pi}{2}, \text{ and } \gamma = \frac{3\pi}{4} \text{ for } U.$$



Pic. 55 Simulated circuit on IBM Qiskit

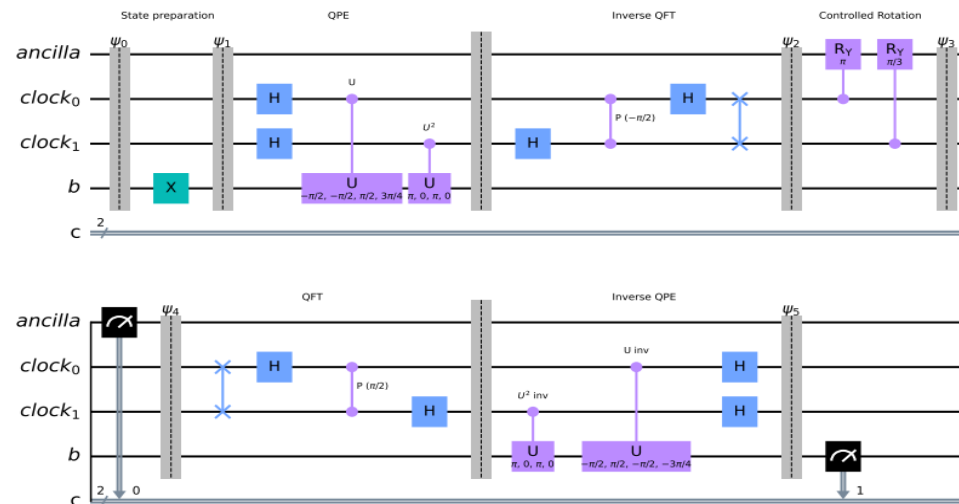
Implementation Methodology

3. Quantum Phase Estimation

- Next, by setting $\theta = \pi$, $\phi = 0$, $\lambda = \pi$, and $\gamma = 0$, we can implement U^2 .
- Applying the inverse of QPE at the end of the algorithm is a simple matter of finding the inverse of each unitary gate and applying it in reverse order. We can clearly see from $U^2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ that U^2 is real and symmetric, hence $(U^2)^{-1} = (U^2)^\dagger = U^2$.
- Next, we can easily find the inverse of U by conjugating and transposing it:
- Hence:

$$U^{-1} = U^\dagger = \frac{1}{2} \begin{pmatrix} -1 - i & -1 + i \\ -1 + i & -1 - i \end{pmatrix}.$$

- We can implement this using the four parameter arbitrary unitary gate by setting $\theta = -\frac{\pi}{2}$, $\phi = \frac{\pi}{2}$, $\lambda = -\frac{\pi}{2}$, and $\gamma = -\frac{3\pi}{4}$.



Pic. 55 Simulated circuit on IBM Qiskit

Implementation Methodology

4. Ancilla Rotation

- At this stage of the algorithm, we wish to encode the inverse of the calculated eigenvalue onto the amplitude of the ancilla register as follows:

$$|0\rangle \rightarrow \frac{1}{\widetilde{\lambda}} |0\rangle.$$

- Such a transformation, however, is not possible. Instead, we apply the following change on the ancilla qubit:

$$|0\rangle \rightarrow \sqrt{1 - \left(\frac{C}{\widetilde{\lambda}}\right)^2} |0\rangle + \frac{C}{\widetilde{\lambda}} |1\rangle,$$

- In order to affect the above change, we must apply the controlled rotation $R_y(\theta_i)$. C must be chosen such that it is not larger than the smallest eigenvalue, so as not to disturb normalization. In our example, we set $C = 1$.
- By comparing the second equation with the representation of an arbitrary quantum state $|\psi\rangle = \cos\frac{\theta}{2} + e^{i\phi} \sin\frac{\theta}{2}$, we can determine the required angle of rotation:

$$\theta_i = 2\arcsin\frac{1}{\widetilde{\lambda}_i}.$$

- Hence, for $\widetilde{\lambda}_0 = 1$, we find that $\theta_0 = \pi$, and the required rotation is $R_y(\pi)$. For $\widetilde{\lambda}_1 = 2$, we find that $\theta_1 = \frac{\pi}{3}$.

Implementation Methodology

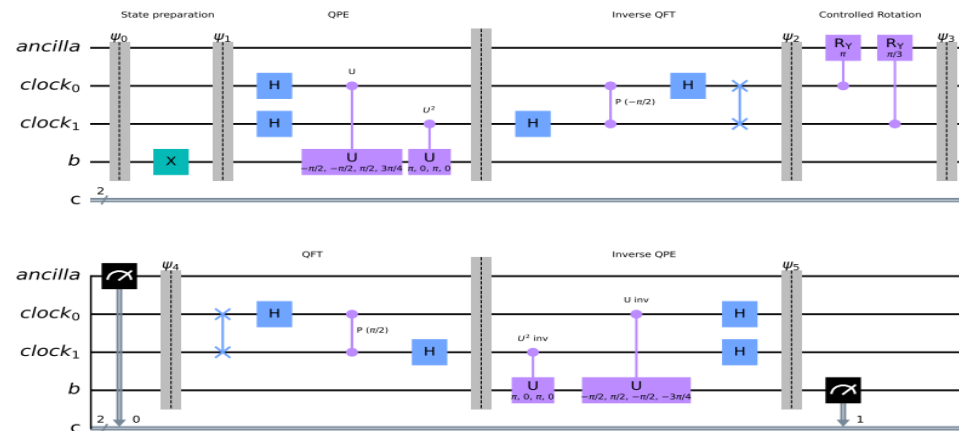
4. Ancilla Rotation

- In effect, we want to apply $|b\rangle = \sum_{i=0}^{N-1} b_i |u_i\rangle$:

$$\Theta(C_1 C_0) = \frac{\pi}{3} C_1 + \pi C_0,$$

where $\Theta(C_1 C_0)$ is a function that takes the values encoded in the clock register encoded in the clock register in binary form, and outputs the required rotation angle θ . This is achieved using two controlled rotation gates as shown in Pic.55.

- In contrast with QPE, the ancilla rotation is an instance of Amplitude Encoding, where the relevant information, i.e., the inverse of the eigenvalue, is encoded into the amplitude of a qubit.

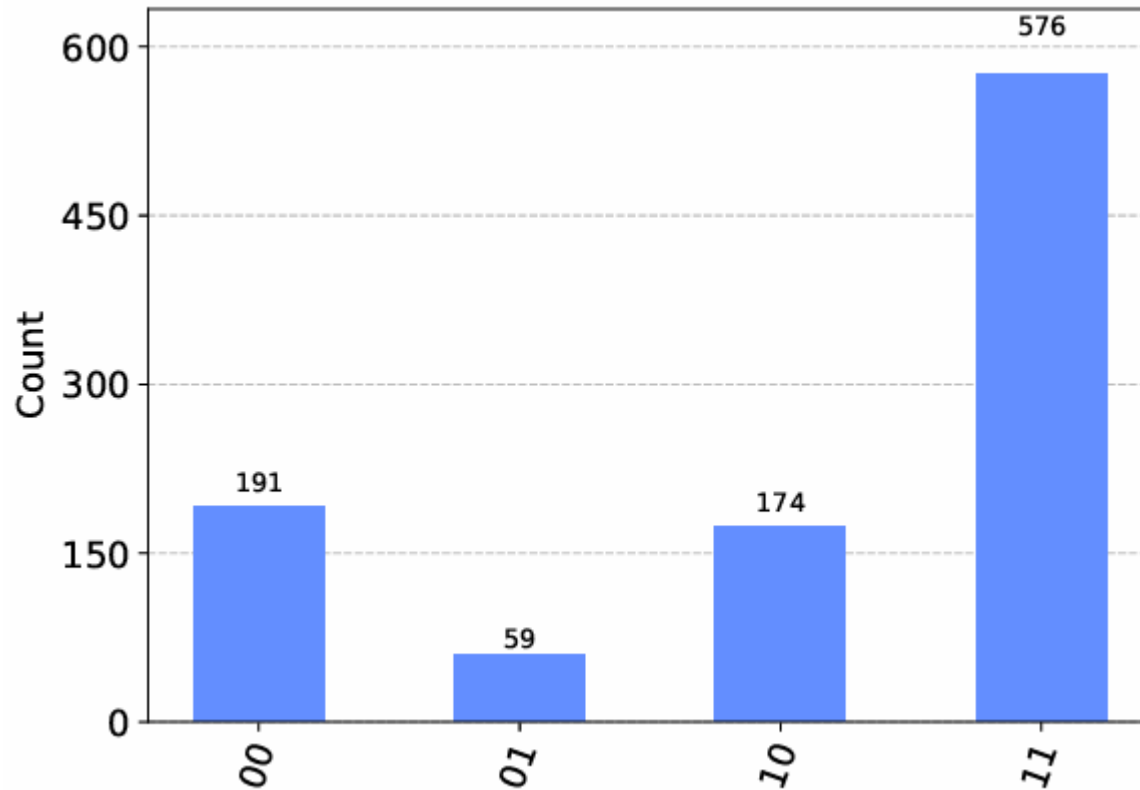


Pic. 55 Simulated circuit on IBM Qiskit

Time and Resource Efficiency

- **Classical Runtimes:** Standard classical methods like Gaussian elimination take $\mathcal{O}(N^3)$ time. However, this drops to $\mathcal{O}(N\sqrt{k})$ if the matrix is sparse and well-conditioned, and you only need a summary statistic.
- **HHL Algorithm Runtime:** This quantum algorithm achieves an exponentially faster runtime of $\mathcal{O}(\frac{K^2 \log N}{\epsilon})$, assuming the matrix is s -sparse (maximum of s non-zero elements per row) and ϵ is the acceptable margin of error.
- **Condition Number (k):** The ratio of the smallest to largest eigenvalue, which indicates the system's sensitivity to perturbations.
- **Quantum Phase Estimation (QPE) Challenges:** QPE is a major source of error and is highly resource-intensive. It requires many qubits and gates, scaling with the system size, and its computational complexity scales as $\mathcal{O}(1/\epsilon)$ based on the desired accuracy of the eigenvalues.
- **Eigenvalue Inversion Challenges:** This stage is also resource-heavy because it relies on a non-unitary transformation, which may require multiple attempts to successfully produce the required $|1\rangle$ state.

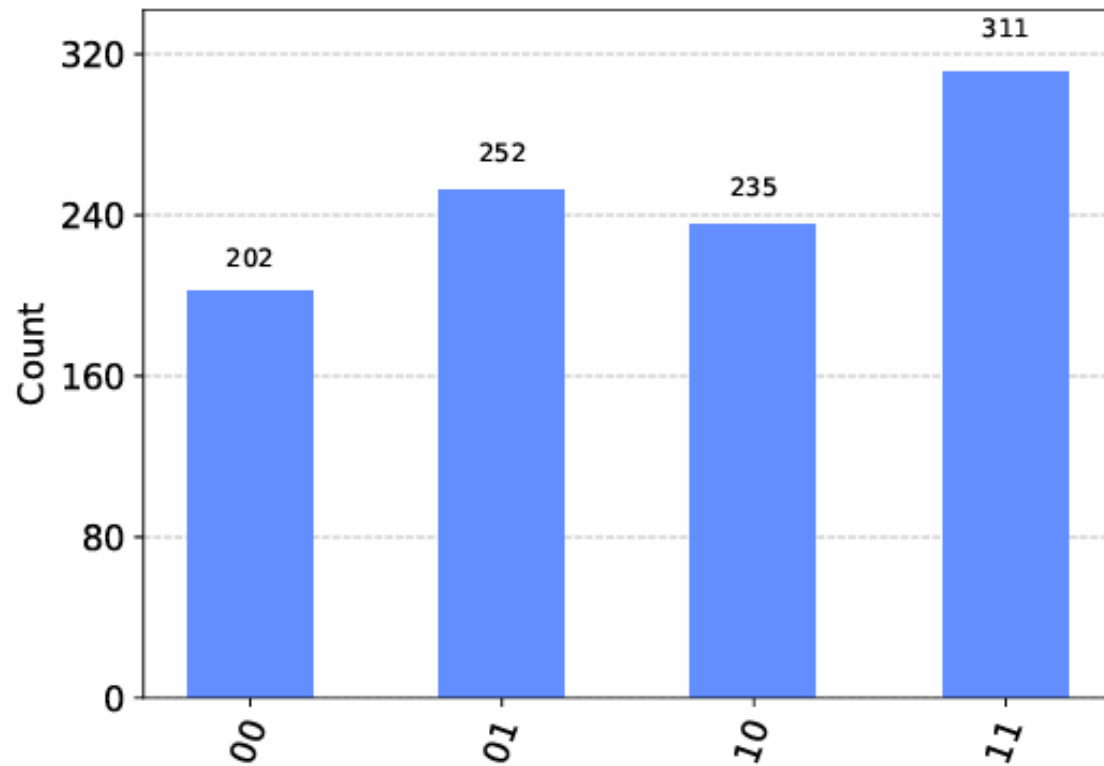
Results



Pic. 56 Simulated results for 1000 shots

- We ran the circuit shown in Pic.55, with our chosen example as input, first on a high performance simulator, and then proceeded to run the same circuit on real quantum hardware.
- The results of the simulation are presented in Pic. 56 with 1000 shots fired. Recall that we are only interested in case where the measurement of the ancilla results in $|1\rangle$. Therefore we look at the bins marked 01 and 11, whose heights correspond to the magnitude squared of the first and second coefficients of the solution vector $|x\rangle$ respectively.
- The calculated ratio thus obtained is 1 : 9.762, which is very close to the true ratio of the solution vector based on the classical solution, i.e., 1 : 9.

Results



Pic. 57 Quantum computer results for 1000 shots

- Running the same circuit on actual quantum hardware generated the results presented in Pic. 57.
- The corresponding ratio in this case is 1 : 1.234 which is significantly different from the true value.
- This difference is due to noise from the environment and decoherence arising from the usage of a real quantum computer as opposed to a simulator.
- The circuit we have implemented can be generalized to some extent, as we can input any 2×1 vector $|b\rangle$ without needing to change other elements of the circuit. (Note that solving for a different matrix A would require significant changes to the circuit with our current implementation methodology).

Results

- From several different vectors and compared the solutions from the simulator to the classical solutions. A summary of the results is presented in the following table.

Table 1: Results for different inputs of vector $|b\rangle$

Input state	Probability Ratio of $ x\rangle$		Relative Error
	Classical	Simulation	
$ +\rangle$	1 : 1	1 : 1.315	31.5%
$ -\rangle$	1 : 1	1 : 0.992	0.8%
$ 0\rangle$	1 : 0.111	1 : 0.119	7.2%
$ 1\rangle$	1 : 9	1 : 9.762	8.5%